

Hybrid MPI and OpenMP Programming

<https://gemini.google.com/app/e94e66f0b5e3168d>

User prompt: Help me structure a hybrid Open MPI + OpenMP program using MPI_Init_thread with MPI_THREAD_FUNNELED. Design a scheme where MPI strides across processes while OpenMP dynamically distributes work inside each process using #pragma omp parallel for schedule(dynamic, 1000) to eliminate thread-level load imbalance and false sharing.

Response: Hybrid Open MPI and OpenMP computing combines distributed-memory coarse partitioning via MPI processes with shared-memory fine-grained multi-threading via OpenMP.

```
#include <mpi.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define N 100000000LL

int main(int argc, char *argv[]) {
    int provided;

    // 1. Initialize MPI with funneled thread support
    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
    if (provided < MPI_THREAD_FUNNELED) {
        fprintf(stderr, "Error: Required MPI_THREAD_FUNNELED level not supported.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // 2. Coarse MPI Domain Decomposition (Process-level chunking)
    long long chunk_per_process = N / size;
    long long start_idx = rank * chunk_per_process;
    long long end_idx = (rank == size - 1) ? N : start_idx + chunk_per_process;
    long long local_size = end_idx - start_idx;

    double *local_data = (double *)malloc(local_size * sizeof(double));

    // 3. Intra-process Dynamic OpenMP Scheduling
    #pragma omp parallel for schedule(dynamic, 1000)
    for (long long i = 0; i < local_size; i++) {
        long long global_i = start_idx + i;
        // Example workload computation
        local_data[i] = (double)(global_i * 2);
    }

    // Accumulation workload
    double local_sum = 0.0;
    #pragma omp parallel for schedule(dynamic, 1000) reduction(+:local_sum)
    for (long long i = 0; i < local_size; i++) {
        local_sum += local_data[i];
    }

    // 4. MPI Communication (Executed strictly by the main thread)
    double global_sum = 0.0;
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Computation complete. Global sum: %f\n", global_sum);
    }

    free(local_data);
    MPI_Finalize();
    return 0;
}
```

Key Structural Considerations

- **Thread Support (MPI_THREAD_FUNNELED):** Informs the MPI runtime that while the application is multi-threaded, only the main thread will make MPI calls. All MPI_* routines must reside outside active #pragma omp parallel blocks or inside master-only sections.
- **Coarse vs. Fine Work Distribution:** MPI splits the array across global nodes in contiguous sub-blocks (start_idx to end_idx). OpenMP then distributes each process's local domain across worker threads.
- **Load Imbalance Mitigation:** #pragma omp parallel for schedule(dynamic, 1000) dynamically hands out chunks of 1,000 iterations to worker threads on demand. If a thread finishes early due to variable computation costs, it immediately grabs the next chunk.

- **False Sharing Prevention:** Setting the chunk size to 1000 ensures that worker threads write to large, contiguous byte sequences (8,000 bytes for double). Because this significantly exceeds standard 64-byte L1/L2 cache lines, worker threads rarely write to the same cache line, effectively preventing false sharing cache-invalidation cycles.

User prompt: Act as a parallel computing expert. Analyze my Open MPI C implementation for searching primes below \$n\$. Evaluate my cyclic partitioning scheme (\$3 + 2r\$ with stride \$2p\$) for residue class lock-in and load imbalance (e.g., odd prime factors \$d \nmid p\$ where \$p_{\text{eff}} = p(1 - 1/d)\$). Suggest an optimized block-cyclic or interleaved chunking scheme to ensure optimal load distribution across all ranks.

Response: Cyclic striding by \$2p\$ locks each MPI process into a fixed residue class modulo \$2p\$, creating severe load imbalance whenever small prime factors divide process count \$p\$. To eliminate this imbalance and preserve memory locality, ranks must process continuous block-cyclic chunks that span all residue classes modulo small primes.

Residue Class Lock-in Analysis

- **Modulo Invariance:** Testing odd numbers via \$x(r, k) = 3 + 2r + 2pk\$ means rank \$r\$ tests numbers where \$x_k \equiv (3 + 2r) \pmod{2p}\$. For any odd prime factor \$d \mid p\$:

$$x(r, k) \equiv (3 + 2r) \pmod{d}$$

- **Workload Disparity:**
 - **Trivial Ranks** (\$3 + 2r \equiv 0 \pmod{d}\$): Every number assigned to rank \$r\$ is a multiple of \$d\$. Trial division aborts on the very first factor \$O(1)\$, leaving the rank virtually idle.
 - **Candidate Ranks** (\$\gcd(3 + 2r, 2p) = 1\$): These ranks process numbers coprime to small factors, requiring heavy trial division up to \$O(\sqrt{x})\$.
- **Efficiency Decay:** When \$d \mid p\$, a fraction \$1/d\$ of ranks are wasted on multiples of \$d\$. The proportion of fully active ranks shrinks by the Euler-like factor:

$$p_{\text{eff}} = p \prod_{d \mid p} \left(1 - \frac{1}{d}\right)$$

Partitioning Strategy Comparison

Strategy	Residue Lock-In	Macro Load Balance (\$O(\sqrt{x})\$ Growth)	Cache Locality
Cyclic (\$2p\$ Stride)	Severe (High)	Excellent	Poor (strided memory)
Pure Block	None	Poor (higher ranks get larger \$x\$)	Excellent
Block-Cyclic Chunking	None	Excellent	Excellent

Optimized Interleaved Block-Cyclic Scheme

Divide odd candidates into contiguous blocks of size \$B\$ (e.g., \$B = 32,768\$ elements to fit L1/L2 cache). Distribute global chunk index \$C\$ to rank \$r\$ when \$C \equiv r \pmod{p}\$.

```
// B: Chunk size (number of odd integers per block)
long long B = 32768;
long long total_odds = (n - 3) / 2 + 1;
long long total_chunks = (total_odds + B - 1) / B;

for (long long C = rank; C < total_chunks; C += size) {
    long long start_idx = C * B;
    long long end_idx = (start_idx + B < total_odds) ? start_idx + B : total_odds;

    // Process contiguous odds in chunk C
    #pragma omp parallel for schedule(dynamic, 1000)
    for (long long i = start_idx; i < end_idx; i++) {
        long long val = 3 + 2 * i;
        if (is_prime(val)) {
            // Count or collect prime
        }
    }
}
```

Why Block-Cyclic Resolves the Bottlenecks

- **Uniform Residue Coverage:** Each block \$B\$ sweeps continuously across consecutive odds, ensuring every rank handles an equal distribution of numbers congruent to \$0 \pmod{3}\$, \$0 \pmod{5}\$, and coprime candidates.

- **Asymptotic Load Balancing:** Interleaving blocks across ranks ensures that no single rank is trapped with only the largest candidate numbers where x trial division costs are highest.

User prompt: Design an experimental timing framework using `MPI_Wtime()` and explicit `MPI_Barrier()` calls to isolate search time, gather communication time ($\kappa(p)$), and root serial processing time (r_s via `qsort` and file writing) without skewing idle rank waiting times into communication costs.

```
Response: #include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int compare_long_long(const void *a, const void *b) {
    long long arg1 = *(const long long *)a;
    long long arg2 = *(const long long *)b;
    if (arg1 < arg2) return -1;
    if (arg1 > arg2) return 1;
    return 0;
}

void run_benchmarked_search(int rank, int size, long long *local_primes, int local_count) {
    double t_start_search, t_end_search;
    double t_start_comm, t_end_comm;
    double t_start_root, t_end_root;

    // -----
    // Phase 1: Pure Search Computation
    // -----
    t_start_search = MPI_Wtime();

    // [ Prime search execution occurs here ]

    t_end_search = MPI_Wtime();
    double t_search_local = t_end_search - t_start_search;

    // -----
    // Synchronize to isolate load imbalance idle time from communication costs
    // -----
    MPI_Barrier(MPI_COMM_WORLD);

    // -----
    // Phase 2: Gather Communication Time ( $\kappa(p)$ )
    // -----
    int *recvcounts = NULL;
    int *displs = NULL;
    long long *global_primes = NULL;
    int total_primes = 0;

    t_start_comm = MPI_Wtime();

    if (rank == 0) {
        recvcounts = (int *)malloc(size * sizeof(int));
    }

    // Gather sizes from all processes
    MPI_Gather(&local_count, 1, MPI_INT, recvcounts, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        displs = (int *)malloc(size * sizeof(int));
        displs[0] = 0;
        for (int i = 1; i < size; i++) {
            displs[i] = displs[i - 1] + recvcounts[i - 1];
        }
        total_primes = displs[size - 1] + recvcounts[size - 1];
        global_primes = (long long *)malloc(total_primes * sizeof(long long));
    }

    // Variable gather of results
    MPI_Gatherv(local_primes, local_count, MPI_LONG_LONG,
                global_primes, recvcounts, displs, MPI_LONG_LONG,
                0, MPI_COMM_WORLD);

    t_end_comm = MPI_Wtime();
    double t_comm_local = t_end_comm - t_start_comm;

    // -----
    // Phase 3: Root Serial Processing Time ( $r_s$ )
    // -----
    double t_root_serial = 0.0;
    if (rank == 0) {
```

```

t_start_root = MPI_Wtime();

// 1. Sort combined output
qsort(global_primes, total_primes, sizeof(long long), compare_long_long);

// 2. Sequential file write
FILE *fp = fopen("primes.txt", "w");
if (fp) {
    for (int i = 0; i < total_primes; i++) {
        fprintf(fp, "%lld\n", global_primes[i]);
    }
    fclose(fp);
}

t_end_root = MPI_Wtime();
t_root_serial = t_end_root - t_start_root;

free(recvcounts);
free(displs);
free(global_primes);
}

// -----
// Metrics Aggregation across Ranks
// -----
double t_search_max, t_comm_max, t_search_min;

MPI_Reduce(&t_search_local, &t_search_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
MPI_Reduce(&t_search_local, &t_search_min, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
MPI_Reduce(&t_comm_local, &t_comm_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

if (rank == 0) {
    double total_wall_time = t_search_max + t_comm_max + t_root_serial;
    double imbalance_ratio = (t_search_max - t_search_min) / t_search_max;

    printf("--- Benchmarking Results (Rank 0) ---\n");
    printf("Search Time (Max): %.6f s\n", t_search_max);
    printf("Comm Time κ(p): %.6f s\n", t_comm_max);
    printf("Root Serial r_s: %.6f s\n", t_root_serial);
    printf("Total Wall Time: %.6f s\n", total_wall_time);
    printf("Search Load Skew: %.2f%%\n", imbalance_ratio * 100.0);
}
}

```

Timing Framework Architecture

- **Barrier Synchronization Placement:** Placing `MPI_Barrier()` after $t_{\text{end_search}}$ forces fast ranks to wait until all processes complete local trial division. This ensures that the time spent waiting for the slowest rank (load imbalance) is absorbed before $t_{\text{start_comm}}$, preventing idle waiting latency from inflating the gather timing $\kappa(p)$.
- **Communication Isolation ($\kappa(p)$):** t_{comm} captures only `MPI_Gather` setup, `MPI_Gatherv` collective data transfers, and root-side buffer allocations. Using `MPI_Reduce(..., MPI_MAX)` captures the critical-path bottleneck across all nodes.
- **Root Serial Isolation (r_s):** $t_{\text{root_serial}}$ is executed solely on rank 0 inside a dedicated timing block. It accurately tracks non-parallelizable bottlenecks ($O(N \log N)$ `qsort` and sequential disk I/O) without affecting remote rank timers.
- **Load Imbalance Metric:** Computing $\frac{t_{\text{search_max}} - t_{\text{search_min}}}{t_{\text{search_max}}}$ quantifies worker starvation without contaminating network metrics.

User prompt: Generate C code logic and benchmark configurations comparing $\$p$ times $t\$$ hybrid setups against pure MPI ($\$p$ processes) and pure OpenMP ($\$t$ threads) at matched total worker counts (e.g., $\$2$ times $7\$$ vs $\$14\$$ processes vs $\$14\$$ threads).

```

Response: #include <mpi.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

bool is_prime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

```

```

int main(int argc, char *argv[]) {
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    long long n = (argc > 1) ? atoll(argv[1]) : 100000000LL;
    int num_threads = omp_get_max_threads();

    // Synchronize processes before clocking
    MPI_Barrier(MPI_COMM_WORLD);
    double t_start = MPI_Wtime();

    long long B = 32768; // Chunk size for L1/L2 locality
    long long total_odds = (n - 3) / 2 + 1;
    long long total_chunks = (total_odds + B - 1) / B;
    long long local_primes = 0;

    // Outer MPI block-cyclic stride across chunks
    for (long long C = rank; C < total_chunks; C += size) {
        long long start_idx = C * B;
        long long end_idx = (start_idx + B < total_odds) ? start_idx + B : total_odds;

        // Inner OpenMP dynamic thread dispatch
        #pragma omp parallel for schedule(dynamic, 1000) reduction(+:local_primes)
        for (long long i = start_idx; i < end_idx; i++) {
            long long val = 3 + 2 * i;
            if (is_prime(val)) {
                local_primes++;
            }
        }
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double t_end = MPI_Wtime();
    double local_time = t_end - t_start;

    double max_time;
    long long global_primes = 0;

    MPI_Reduce(&local_time, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&local_primes, &global_primes, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        // Output format: Processes, Threads, Total Workers, Total Primes, Max Time (s)
        printf("%d,%d,%d,%lld,%.6f\n", size, num_threads, size * num_threads, global_primes, max_time);
    }

    MPI_Finalize();
    return 0;
}

```

Benchmark Configuration Matrix ($W = 14$ Workers)

Paradigm	MPI Ranks (p)	OMP Threads (t)	Total Workers (W)	Primary Bottleneck Risk
Pure MPI	14	1	14	Memory overhead, MPI process context switching
Pure OpenMP	1	14	14	NUMA cross-socket latency, lock contention
Balanced Hybrid	2	7	14	Thread launch overhead across sockets
Asymmetric Hybrid	7	2	14	Over-decomposed MPI domains vs thread pool size

Execution Commands & Thread Binding

Export environment settings and run the binary using strict hardware locality controls to prevent socket migration during tests:

- **Pure MPI Configuration (14×1):**

```

export OMP_NUM_THREADS=1
mpirun -np 14 --bind-to core ./prime_bench 100000000

```

- **Pure OpenMP Configuration (1×14):**

```

export OMP_NUM_THREADS=14
export OMP_PROC_BIND=true

```

```
export OMP_PLACES=cores
mpirun -np 1 --bind-to socket ./prime_bench 100000000
```

- **Hybrid Configuration (2×7):**

```
export OMP_NUM_THREADS=7
export OMP_PROC_BIND=close
export OMP_PLACES=cores
mpirun -np 2 --map-by numa --bind-to numa ./prime_bench 100000000
```

Comparative Evaluation Metrics

- **NUMA Domain Overhead:** Map 1 MPI process per NUMA socket to restrict thread migration while eliminating remote socket memory access penalties inherent in Pure OpenMP setups.
- **Memory Footprint:** Monitor per-node memory utilization; Pure MPI duplicates communicator buffers and process structures across 14 ranks, whereas Hybrid shares process buffers among 7 threads.
- **Collective Scaling ($\kappa(p)$):** Track communication phase latencies; Hybrid setups reduce the participant process count in collective calls from 14 down to 2 ranks.

User prompt: Draft a concise 6-to-7-minute presentation script covering methodology, phase breakdowns, speedup plateaus, limitations (e.g., single-node shared memory constraints), and future multi-node CAAS cluster scaling.

Response: [0:00 – 1:30] Slide 1: Introduction & Hybrid Methodology

Good morning/afternoon, everyone. Today, I'm presenting our hybrid parallel computing framework designed for high-performance prime candidate searching, combining coarse-grain Open MPI process partitioning with fine-grain OpenMP dynamic multi-threading.

Standard cyclic domain partitioning—like striding by $2p$ across odd numbers—suffers from severe residue-class lock-in. When the process count p shares prime factors like 3 or 5, specific ranks get stuck exclusively processing numbers congruent to those multiples. These "trivial" ranks abort trial division almost instantly in $O(1)$ time, while non-trivial ranks bear the entire computational burden of deep trial division up to $O(x)$. To solve this, we implemented an interleaved block-cyclic chunking scheme. By processing contiguous blocks of $B = 32,768$ elements, every rank sweeps across all residue classes uniformly, completely eliminating structural load skew.

At the intra-process level, we initialize MPI using `MPI_THREAD_FUNNELED`, restricting all network operations to the main thread. Inside each process, we dispatch work using `#pragma omp parallel for schedule(dynamic, 1000)`. Assigning dynamic chunks of 1,000 iterations achieves two crucial goals: it balances varying trial-division times across thread pools, and its 8,000-byte allocation size vastly exceeds standard 64-byte L1/L2 cache lines, completely eliminating false sharing.

[1:30 – 3:00] Slide 2: Execution Phase Breakdown & Benchmark Isolation

To rigorously analyze where execution time is spent, our experimental timing framework uses explicit `MPI_Barrier()` syncs and `MPI_Wtime()` calls to decouple computation, communication, and serial overheads into three isolated phases:

1. **Parallel Search Phase (t_{search}):** Ranks independently perform localized prime searches using our block-cyclic distribution.
2. **Gather Communication Phase ($\kappa(p)$):** An explicit `MPI_Barrier()` forces fast processes to wait for the slowest worker *before* initiating `MPI_Gatherv`. This guarantees that idle times from residual computational load skews are fully absorbed into search metrics and never wrongly misclassified as network latency.
3. **Root Serial Processing Phase (r_s):** Rank 0 performs sequential $O(N \log N)$ sorting via `qsort` and writes output results to disk.

By tracking $t_{\text{search_max}}$, $t_{\text{comm_max}}$, and r_s independently, we can quantify exact communication overheads and load imbalance ratios without metric contamination.

[3:00 – 4:30] Slide 3: Speedup Plateaus & Hardware Bottlenecks

When comparing matched-worker configurations—such as 14 Pure MPI processes vs. 14 Pure OpenMP threads vs. a 2×7 Hybrid configuration—we observe distinct scaling plateaus governed by hardware boundaries and Amdahl's Law.

- **Pure OpenMP (1×14)** suffers at high core counts due to NUMA cross-socket memory latencies and cache coherency traffic across CPU socket interconnects.
- **Pure MPI (14×1)** delivers strong search performance but hits memory footprint constraints and higher collective aggregation overhead during `MPI_Gatherv` operations.
- **Hybrid (2×7)** balances these trade-offs by pinning 1 MPI process per NUMA domain and distributing 7 OpenMP threads locally within each socket's shared L3 cache.

However, as worker counts scale, overall speedup plateaus. This is driven primarily by Amdahl's Law: the serial processing phase r_s on Rank 0—specifically single-threaded `qsort` and sequential disk I/O—becomes the dominant bottleneck as parallel compute time drops toward zero.

[4:30 – 6:30] Slide 4: Limitations & Future CaaS Cluster Scaling

Our current architecture faces a core single-node shared-memory limitation: scaling worker counts within a single machine eventually saturates memory bus bandwidth and bus request queues.

To break past these single-node limits, our target future architecture involves scaling to multi-node CaaS (Container-as-a-Service) HPC clusters. In this model, containerized workloads will be deployed across independent physical nodes, mapping 1 MPI process per container/NUMA node with local OpenMP thread pinning.

To ensure true linear scaling in multi-node environments, we will upgrade our framework to resolve the serial r_s bottleneck:

- Replace single-node `qsort` with a distributed parallel merge-sort across all MPI ranks.
- Replace sequential file writes with collective `MPI-IO` (`MPI_File_write_at_all`), allowing ranks to stream prime output concurrently directly into shared persistent storage.

Thank you. I am now open to any questions.

User prompt: Act as a Monash University lab assessor for FIT3143. Ask me 4 challenging Q&A questions probing load imbalance, residue congruence, shared-memory vs message-passing overheads, and Amdahl's law parameter derivations. Evaluate my responses strictly against the HD rubric for correct use of parallel computing terminology (e.g., inter-process communication, lock contention, thread funneled, cache line invalidation, core oversubscription).

Response: Welcome to your FIT3143 viva assessment. To achieve a High Distinction (HD) mark, your answers must demonstrate rigorous mathematical derivations, precise architectural reasoning, and exact parallel computing terminology—including concepts like inter-process communication, lock contention, thread funneled runtime semantics, cache line invalidation, NUMA domain affinity, and core oversubscription.

Answer the following four assessment questions:

1. Residue Congruence & Load Imbalance Mathematically prove why cyclic partitioning ($3 + 2r$ with stride $2p$) causes residue class lock-in whenever an odd prime factor d divides process count p . Explain how this causes asymptotic complexity degradation between trivial ranks ($O(1)$) and candidate ranks ($O(x)$), and demonstrate why interleaved block-cyclic chunking ($B = 32, 768$) mathematically resolves this skew.

2. Shared-Memory vs. Message-Passing Architectural Overheads Compare three matched-worker configurations ($W = 14$): Pure MPI (14×1), Pure OpenMP (1×14), and Hybrid (2×7). Evaluate each against:

- Inter-process communication memory footprint vs. shared-memory thread synchronization overhead.
- NUMA cross-socket memory access latency and cache line invalidation.
- The risk of core oversubscription and thread migration when binding worker threads using `OMP_PROC_BIND` and `OMP_PLACES`.

3. Amdahl's Law Parameter Derivation Given measured parallel search time $t_{\text{search}}(p)$, collective communication time $\kappa(p)$, and root serial processing time r_s (sorting and I/O):

- Derive the formula for total speedup $S(p)$ relative to single-core execution $T(1)$.
- Derive the implicit serial fraction f as a function of r_s , $t_{\text{search}}(1)$, and $\kappa(p)$.
- Calculate the theoretical maximum speedup limit $S_{\text{max}} = \lim_{p \rightarrow \infty} S(p)$ and explain how distributed `MPI-IO` modifies this upper bound.

4. Thread Runtime Semantics & OpenMP Scheduling Mechanics Explain the operational differences between `MPI_THREAD_FUNNELED`, `MPI_THREAD_SERIALIZED`, and `MPI_THREAD_MULTIPLE`. What specific runtime errors or performance penalties occur if an MPI collective call is executed within an un-gated `#pragma omp parallel` region under `MPI_THREAD_FUNNELED`? Furthermore, explain why `#pragma omp parallel for schedule(dynamic, 1000)` eliminates both thread-level load starvation and false sharing at the L1/L2 cache line level.

Please provide your complete answers to all four questions when you are ready to begin your evaluation.
